

Projet *Réservation de salles* : Conception v1.0

1. Table des matières

Projet <i>Réservation de salles</i> : Conception v1.0	1
1. Table des matières	1
2. Objectif du document	2
3. Architecture	2
3.1. Choix des technologies	2
3.2. Contraintes techniques	3
4. Technologies utilisées	3
4.1. Serveur web	3
4.2. Stockage des données	3
4.3. Couche de persistance	3
4.4. Couche métier	3
4.5. Couche service et API REST	3
4.6. Couche présentation	4
4.7. Authentification	4
4.8. Environnement de développement	4
4.9. Tests	4
4.10. Packages et dépendances	4
4.11. Déploiement	5
5. Préliminaire à la conception	6
6. Cas d'utilisation	7
6.1. Rappel : modèle trouvé en analyse	7
6.2. Groupe 1 : Gestion des comptes	7
6.3. Groupe 2 : Gestion des salles	8
6.4. Groupe 3 : Gestion des réservations	8
7. Regroupement des classes	8
7.1. Groupe domaine	8
7.2. Groupe repositories	8
7.3. Groupe services	8
7.4. Groupe contrôleurs REST	8
8. Choix, questions ouvertes et remarques	9
9. Annexes	9
9.1. Terminologie	9
9.2. Bibliographie	9

2. Objectif du document

Ce document aborde l'architecture, la conception et les choix techniques pour l'implémentation du projet « Réservation de salles ». Les diagrammes suivent le langage de modélisation UML et la méthodologie Arrington.

- On commencera par énumérer les diverses contraintes techniques qui pèsent sur notre projet ;
- on décrira ensuite les technologies choisies ;
- puis l'architecture (les deux étant évidemment liés) ;
- et enfin, nous décrirons le design de notre système en revenant sur les *use cases*.

3. Architecture

3.1. Choix des technologies

Le système est découpé en deux parties distinctes : un **backend** Spring Boot exposant une API REST, et un **frontend** Vue.js qui consomme cette API. Ce choix découple clairement la logique métier de la présentation, ce qui facilite la maintenance et l'évolution indépendante de chaque partie.

Les points à considérer sont en particulier :

- la complexité de l'interface utilisateur ;
- les contraintes de déploiement ;
- le nombre et le type d'utilisateurs ;
- la sécurité ;
- les performances et le passage à l'échelle.

3.1.1. Administration du système

- opérations essentiellement CRUD (gestion des comptes, des salles) ;
- l'administrateur est un utilisateur interne ;
- sécurité importante : accès réservé aux administrateurs.

Une interface web classique est suffisante. La même application Vue.js pourra proposer des vues spécifiques aux administrateurs, protégées par le système d'authentification.

3.1.2. Côté utilisateurs

- les utilisateurs consultent et filtrent la liste des salles disponibles ;
- ils effectuent, modifient et annulent des réservations ;
- ils reçoivent des notifications par mail ;
- une interface web responsive est souhaitée (mobile inclus).

L'application Vue.js fournit une interface fluide sans rechargement de page complet. Les interactions avec l'API REST permettent de filtrer les salles dynamiquement et de gérer les réservations en temps réel.

3.1.3. Côté responsables

- les responsables gèrent les salles, les équipements et les plages de disponibilité ;
- ils valident ou rejettent les demandes de réservation ;
- l'interface est plus riche que celle des utilisateurs simples, mais reste dans les capacités d'une SPA Vue.js.

3.2. Contraintes techniques

- le système doit être accessible de l'extérieur via HTTPS ;
- le backend expose une API REST/JSON ; le frontend Vue.js est une application distincte qui consomme cette API ;
- le système doit être fiable pour l'envoi de mails (confirmation de réservation, annulation) ;
- l'application doit être raisonnablement sécurisée (authentification JWT, HTTPS, validation des données côté serveur).

Deux architectures de déploiement sont envisagées, décrites en détail en section 4.11 :

- **Architecture simple** (cible initiale) : un seul conteneur backend, un seul conteneur frontend, une seule base de données PostgreSQL. Suffisante pour un établissement d'enseignement à charge modérée. C'est cette architecture qui sera mise en place en premier.
- **Architecture avec load balancer** (évolution possible) : si la charge augmente, on peut passer à une architecture horizontalement scalable avec un load balancer devant plusieurs instances du backend. L'utilisation de **tokens JWT sans état** (*stateless*) facilite cette évolution : aucune session côté serveur n'est à partager entre les instances.

4. Technologies utilisées

4.1. Serveur web

Pour des raisons de facilité de maintenance, on choisit d'utiliser une application **Spring Boot** hébergée dans un conteneur Docker. Le serveur utilisé sera un serveur **Tomcat embarqué**. Le backend expose uniquement des endpoints REST (pas de rendu de templates côté serveur).

4.2. Stockage des données

Les données seront stockées dans une base **PostgreSQL**, aussi bien en développement qu'en production. Ce choix garantit la cohérence entre les environnements et évite les surprises liées aux différences de comportement entre bases de données.

4.3. Couche de persistance

La couche de persistance sera **JPA / Hibernate**, que l'équipe maîtrise bien. Les entités seront annotées JPA, et les repositories seront des interfaces Spring Data JPA.

4.4. Couche métier

La couche métier sera composée d'entités JPA et d'implémentations de services Spring (**@Service**). Les règles métier importantes (vérification de disponibilité, gestion des conflits de réservation, transitions d'état) seront encapsulées dans les entités ou dans des classes de service dédiées.

4.5. Couche service et API REST

La couche service est isolée de la couche présentation via des **DTOs**. Les **contrôleurs REST** (**@RestController**) exposent les endpoints JSON, et délèguent le traitement aux services. Cette séparation permet de tester les services indépendamment des contrôleurs.

4.6. Couche présentation

La couche présentation est entièrement gérée par **Vue.js**, une application SPA (Single Page Application). Elle communique avec le backend via des appels HTTP (Axios ou Fetch). Le build Vue.js produit des fichiers statiques (HTML, CSS, JS) qui sont servis par **Nginx** directement, indépendamment du backend Spring Boot. Ce choix est cohérent avec l'architecture horizontalement scalable retenue : les instances Spring Boot ne gèrent que l'API REST, et Nginx reste le seul point de distribution du frontend.

4.7. Authentification

L'authentification sera gérée par **Spring Security** avec des **tokens JWT** (JSON Web Tokens). Le frontend envoie ses identifiants, reçoit un token, et l'inclut dans l'en-tête **Authorization** de chaque requête. Ce mécanisme est adapté à une architecture découplée frontend/backend.

On pourra envisager dans un second temps l'utilisation de **Keycloak** pour une gestion plus avancée des identités.

4.8. Environnement de développement

Les projets backend seront manipulés via **Gradle**. Le projet frontend utilisera **npm** avec **Vite** comme outil de build. Les développeurs utilisent l'IDE de leur choix.

4.9. Tests

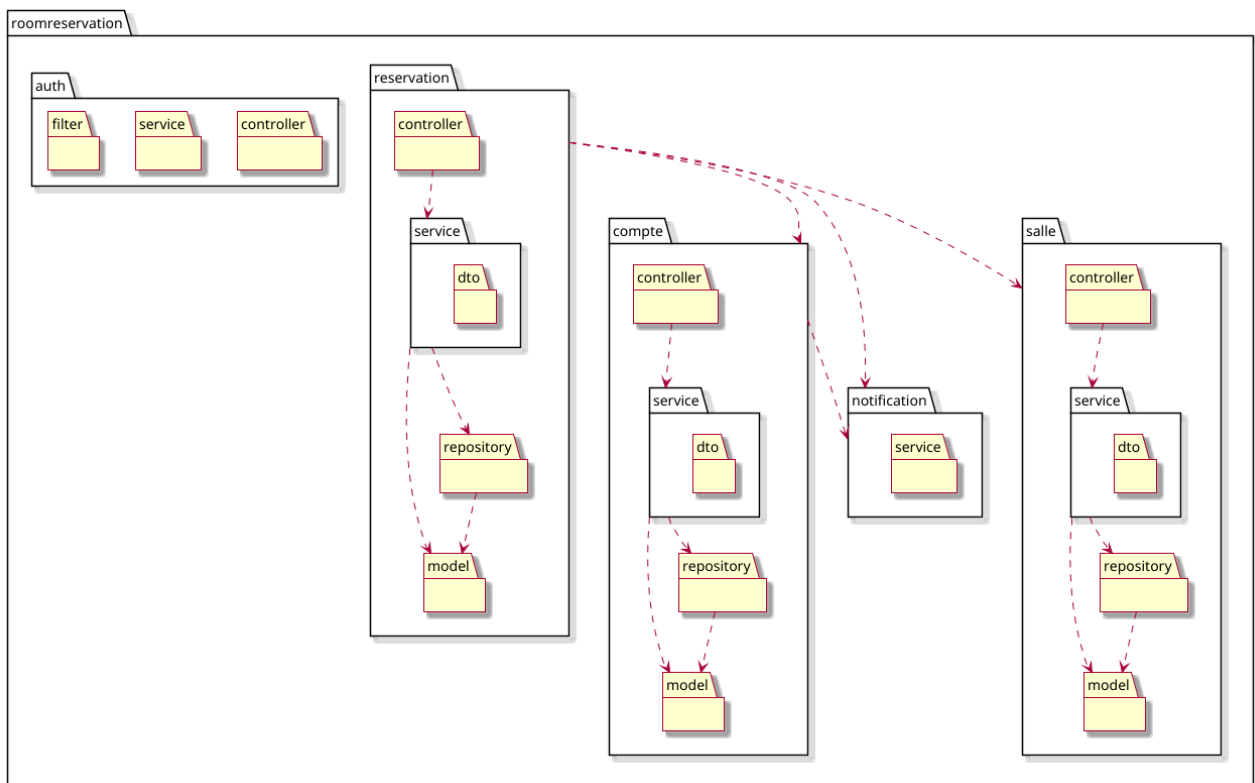
- Tests unitaires backend : **JUnit 5** avec **Mockito** pour les mocks ;
- Tests d'intégration : **Spring Boot Test** avec base PostgreSQL (via **Testcontainers**) ;
- Tests frontend : **Vitest** (ou Jest) pour les composants Vue.

4.10. Packages et dépendances

L'approche **Acteur/Action** permet de proposer les composants backend suivants :

- **compte** : gestion des demandes de création de compte, validation, refus, et gestion des informations utilisateur ;
- **salle** : gestion des salles, des équipements et des plages de disponibilité ;
- **reservation** : cycle de vie des réservations (création, validation, rejet, annulation, liste d'attente) ;
- **notification** : envoi de mails (confirmation de réservation, annulation, validation de compte) ;
- **auth** : authentification et gestion des tokens JWT.

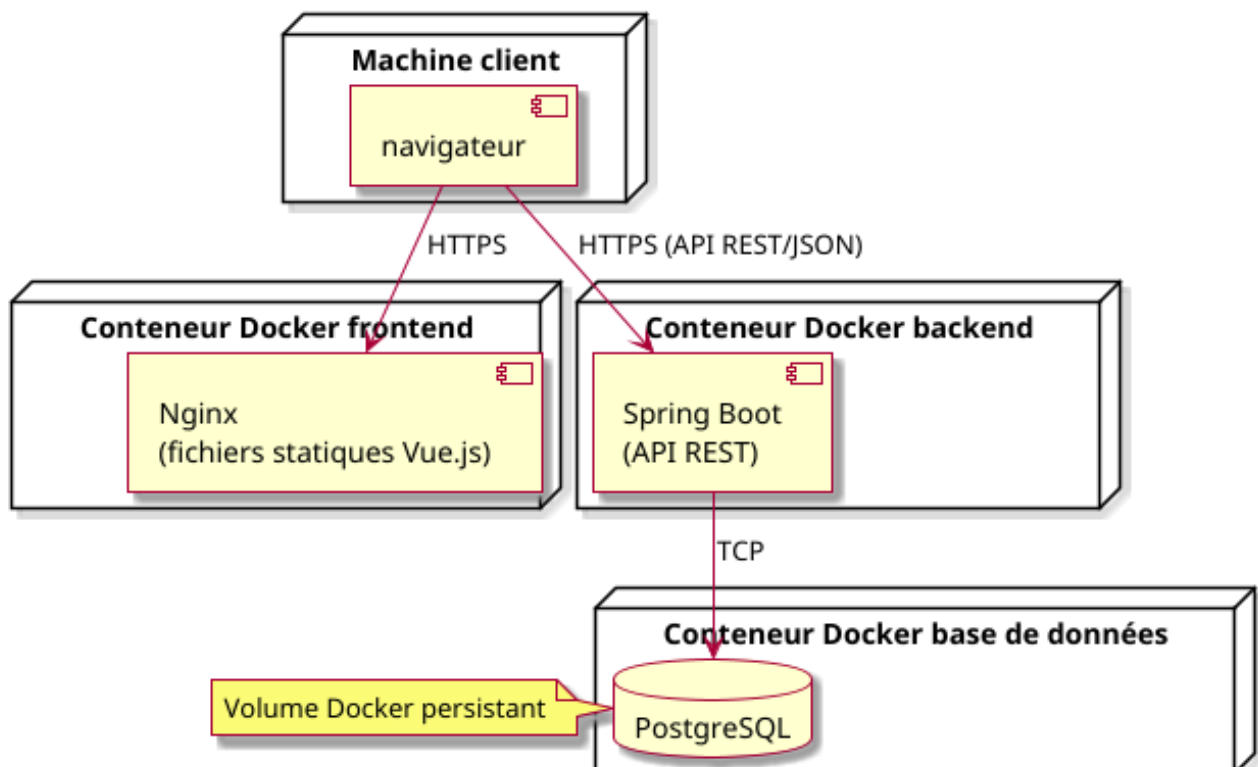
Chaque composant suit la même structure interne :



À chaque composant correspond un (ou plusieurs) `@RestController` qui sert de **façade** et communique avec la couche de présentation à travers des **DTOs**. Les entités JPA ne sont jamais exposées directement dans l'API.

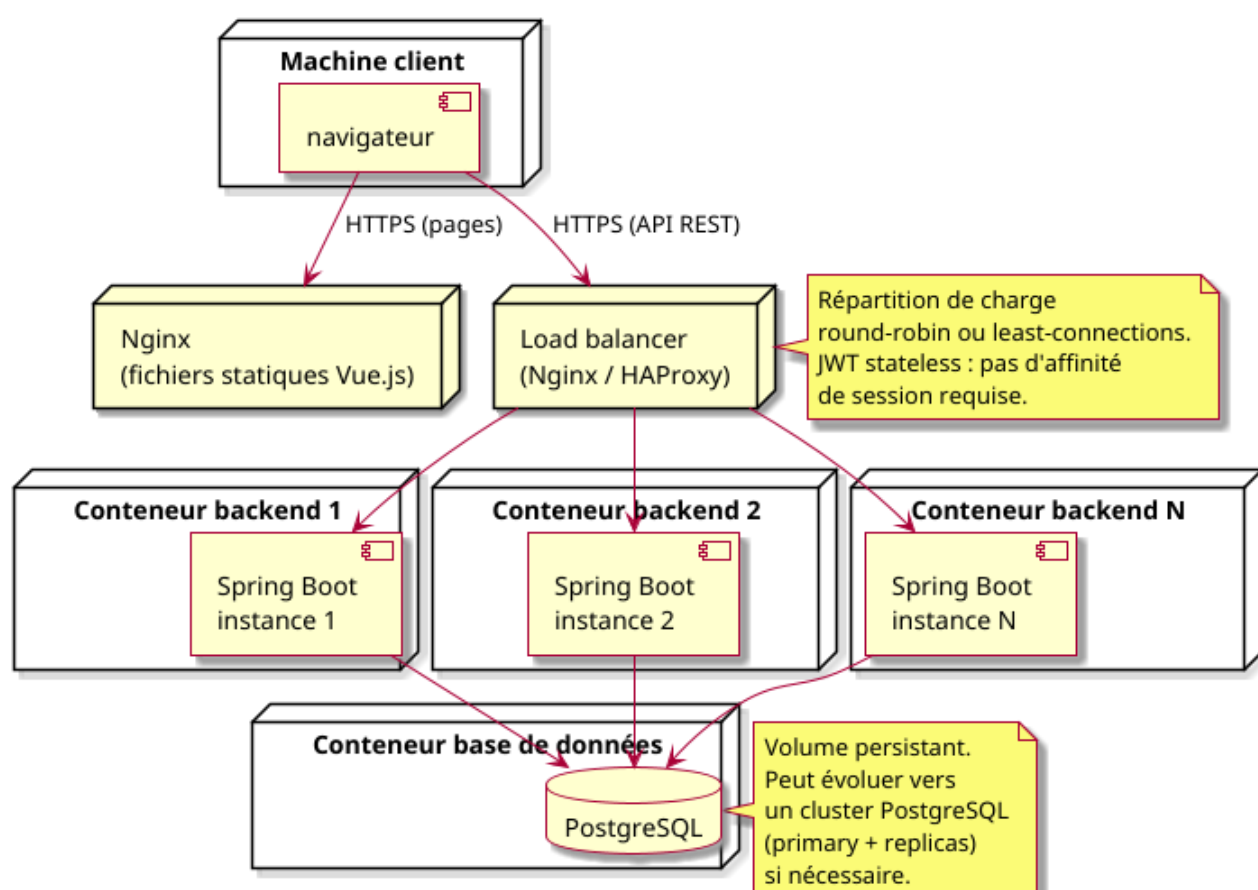
4.11. Déploiement

4.11.1. Architecture simple (cible initiale) Un seul conteneur par composant : frontend, backend, base de données. Orchestration via **Docker Compose**.



- le frontend Vue.js est servi par **Nginx** comme fichiers statiques ;
- le backend Spring Boot expose l'API REST sur un port dédié ;
- la base de données PostgreSQL est dans son propre conteneur avec un volume persistant ;
- les trois conteneurs sont orchestrés via **Docker Compose**.

4.11.2. Architecture avec load balancer (évolution horizontale) Si la charge augmente, on peut multiplier les instances backend derrière un **load balancer**. Le frontend reste servi par un **Nginx** unique — les fichiers statiques Vue.js n'ont pas besoin d'être scalés. L'authentification **JWT** (*stateless*) rend cette évolution naturelle : aucune session serveur à synchroniser entre les instances.



- le frontend Vue.js est servi par un **Nginx** unique — les fichiers statiques étant identiques pour tous les utilisateurs, il n'y a aucun intérêt à les dupliquer ;
- le **load balancer** distribue les appels API REST entre les instances Spring Boot ;
- les tokens **JWT étant sans état**, n'importe quelle instance peut traiter n'importe quelle requête sans partage de session ;
- la base de données PostgreSQL reste centralisée ; elle peut évoluer vers un cluster (primary + replicas en lecture) si elle devenait un goulot d'étranglement ;
- cette architecture peut être gérée avec **Docker Swarm** ou **Kubernetes** selon les besoins.

5. Préliminaire à la conception

La conception est un processus itératif. Avant de traiter les *use cases* un par un, il est utile de faire un premier passage sur les classes métier identifiées en analyse, afin d'anticiper les difficultés.

Les entités principales identifiées lors de l'analyse sont :

- **Compte** : représente un compte utilisateur dans le système ;
- **DemandeCreationCompte** : représente une demande de création de compte en attente de validation ;
- **Salle** : représente une salle pouvant être réservée ;
- **Equipement** : représente un équipement disponible dans une salle ;
- **PlageDisponibilite** : représente une plage horaire durant laquelle une salle est disponible ;
- **Reservation** : représente une réservation effective ou une demande de réservation ;
- **ListeAttente** : représente une position dans la liste d'attente pour une salle.

Un point important à noter : la gestion de la **disponibilité des salles** et la **détection des conflits de réservation** constituent la complexité principale de ce projet. Il faudra traiter ces *use cases* en priorité pour valider le modèle.

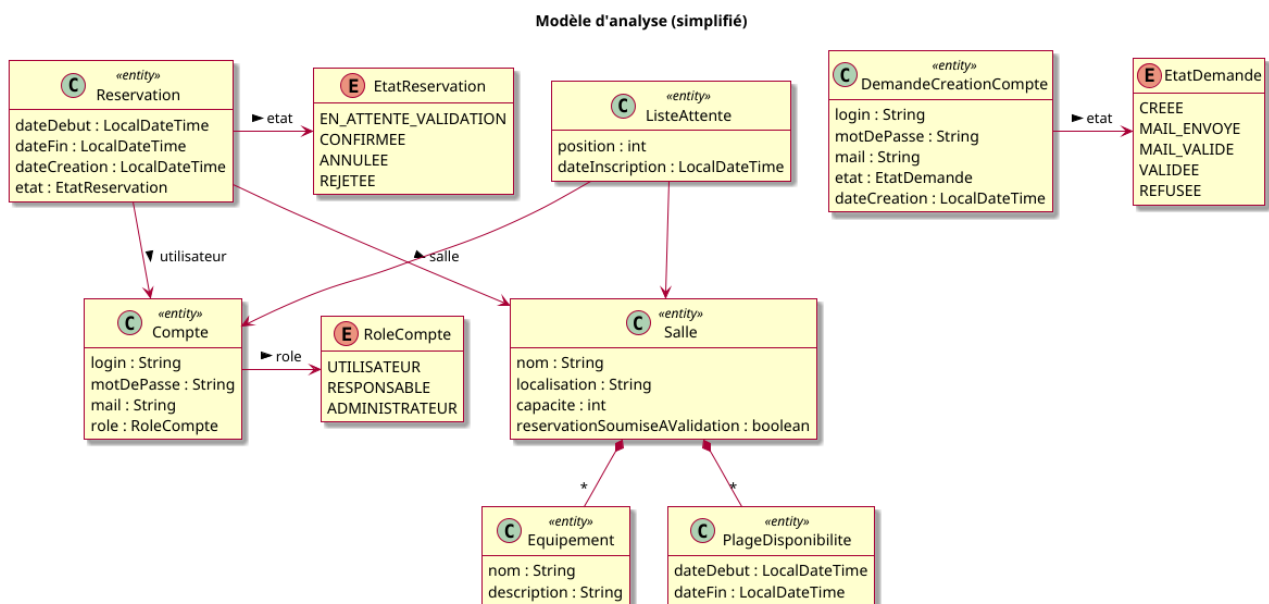
Un second point concerne le **double mode de réservation** : certaines salles sont réservées directement (premier arrivé, premier servi avec liste d'attente), d'autres nécessitent la validation d'un responsable. Ce choix influe sur le cycle de vie de la **Reservation** et son état.

6. Cas d'utilisation

On choisit de développer un sous-ensemble cohérent des fonctionnalités, en commençant par les cas prioritaires identifiés en analyse.

6.1. Rappel : modèle trouvé en analyse

Le modèle issu de l'analyse est un point de départ qui sera affiné pendant la conception.



6.2. Groupe 1 : Gestion des comptes

6.2.1. Déposer une demande de création de compte (à compléter)

6.2.2. Consulter les demandes de création de compte en attente (à compléter)

6.2.3. Valider une demande de création de compte *(à compléter)*

6.2.4. Refuser une demande de création de compte *(à compléter)*

6.3. Groupe 2 : Gestion des salles

6.3.1. Créer une salle *(à compléter)*

6.3.2. Créer un équipement *(à compléter)*

6.3.3. Ajouter une plage de disponibilité d'une salle *(à compléter)*

6.3.4. Consulter la liste des salles *(à compléter)*

6.4. Groupe 3 : Gestion des réservations

6.4.1. Rechercher une salle selon différents critères *(à compléter)*

6.4.2. Réserver une salle *(à compléter)*

6.4.3. Consulter une réservation *(à compléter)*

6.4.4. Annuler une réservation *(à compléter)*

6.4.5. Valider une demande de réservation *(à compléter)*

6.4.6. Rejeter une demande de réservation *(à compléter)*

7. Regroupement des classes

7.1. Groupe domaine

(à compléter)

7.2. Groupe repositories

(à compléter)

7.3. Groupe services

(à compléter)

7.4. Groupe contrôleurs REST

(à compléter)

8. Choix, questions ouvertes et remarques

- **Gestion des conflits de réservation** : la vérification qu'une salle est libre sur un créneau donné est une requête potentiellement complexe. Il faudra définir précisément la requête JPA ou SQL correspondante.
- **Double mode de réservation** : le fait qu'une salle puisse être soit à réservation directe, soit à validation, implique deux branches dans le cycle de vie de **Reservation**. Le *Design Pattern State* pourrait être envisagé pour gérer proprement ces deux comportements.
- **Liste d'attente** : la gestion de la liste d'attente (promotion automatique en cas d'annulation) est un mécanisme à préciser. Elle pourrait être traitée par un événement Spring déclenché lors de l'annulation d'une réservation.
- **Annulation automatique** : quand une salle devient indisponible (travaux, événement prioritaire), les réservations existantes doivent être annulées automatiquement et les utilisateurs notifiés.
- **JWT et sécurité** : il faudra préciser la durée de vie des tokens, le mécanisme de refresh, et les règles CORS pour autoriser les appels depuis le frontend Vue.js.

9. Annexes

9.1. Terminologie

Terme	Définition
Salle	Espace physique pouvant être réservé par les utilisateurs
Plage de disponibilité	Créneau horaire durant lequel une salle peut être réservée
Réservation	Occupation d'une salle sur un créneau par un utilisateur
Responsable	Utilisateur ayant les droits de gestion des salles et de validation des réservations
Administrateur	Utilisateur ayant les droits de gestion des comptes

9.2. Bibliographie

- Avram A., Marinescu F. *Domain-Driven Design Quickly: A Summary of Eric Evans' Domain-Driven Design* [En ligne]. C4Media, 2006. (Enterprise Software Development Series). [lien](#)
- Evans E. *Domain-driven design: tackling complexity in the heart of software*. Boston : Addison-Wesley, 2004.
- Evans E. *Domain-driven design reference: definitions and pattern summaries*. Indianapolis, 2015. [lien](#) (résumé des patterns)
- Gandhi, Raju, Mark Richards, and Neal Ford. 2024. *Head First Software Architecture*. First edition. Head First. O'Reilly Media, Inc.
- Martin R. C. *Clean architecture: a craftsman's guide to software structure and design*. London, England : Prentice Hall, 2018.
- [Projet Cargo](#) un projet qui implémente l'exemple du livre d'Evans
- [Domain Driven Design and Development In Practice](#)